Maddy St. Charles
Climate 410, Lab 3
Due Monday, October 26, 2026 at 5pm

## Methodology

To investigate the energy balance of the earth's atmosphere, I created three different functions that combine linear algebra techniques with atmospheric equations to model the relationship between surface temperature and various characteristics of the atmosphere and planet. A description of each function is listed below, as well as their implementation to create various figures displaying the results.

atm_flux( )

The first function I wrote is atm_flux( ). The parameters for this function are a specified number of layers (nlayers) and the amount of incoming solar radiation (F_solar). There is also an added debugging feature (debug) that can be set to True when calling atm_flux( ) to investigate if the logic in the code is faulty or not.

This function uses linear algebra in the form Ax=b to solve the equations that represent the energy balance of the earth's atmosphere. First, the matrices A and b are initialized to be grids of zeroes, with their sizes dependent on the number of layers called (so, A is a matrix nlayers+1 x nlayers+1 in size, and b is a matrix nlayers+1 in size). In lecture, we found a pattern in the matrices for this atmospheric application where the diagonal values of A are always -2, so I implemented that into the matrix using np.fill_diagonal( ). Additionally, we discovered in lecture that the two diagonals surrounding the center diagonal are values of 1, so I added that information using the same np.fill_diagonal( ), but with the proper indexing. There is one exception to these patterns, though, and that's the first element of the A matrix (which is equal to -1), so I manually went in and changed that value to match. The final thing to initialize is the b matrix, and all that I had to alter for that is the first element, which is the negative value of the incoming solar radiation, -F_solar.

Once the matrices were ready for calculations, I was able to simply pass both A and b into a built-in numpy function to calculate the fluxes for each layer of the atmosphere and the surface of the earth. These results are returned as an array, but in order to make the data easier to interpret and organize, I changed this array into a dictionary using a for loop that assigns each index of the array a label corresponding to its atmospheric layer (F_a{i}). I also assigned the very first element in the array the surface flux label. Finally, I coded the debug feature as demonstrated in the lab manual. When the function is called and debug=True, it will loop through each element in the matrix or array and print its index and assigned value. This will reveal if there are any inconsistencies in the code in comparison to what we expect in real life. In the end, this function returns the dictionary containing the fluxes for the earth's surface and each layer of the atmosphere.

flux2temp( )

This function is very simple: its purpose is to convert the dictionary of fluxes returned from atm_flux( ) into temperatures. As such, its only parameter is that dictionary of fluxes. The function loops through each value in the dictionary and converts it to a temperature using a rearranged version of the Stefan-Boltzmann equation, as given below:

$$T = \sqrt[4]{\frac{F}{\sigma}}$$

As the loop runs and calculates the corresponding temperatures, it also modifies the dictionary labels so that rather than an element being labeled F_a1 for flux, for example, it would now be labeled T_a1 for temperature. These new values and labels are added to a new dictionary, which is then returned.

atm_flux_temp( )

The final function I wrote is very similar to atm_flux( ), except that this one has more parameters that take various characteristics of the atmosphere and planet into account. Additionally, this function converts the fluxes to temperatures itself, so flux2temp( ) is not needed. The two (three, counting debug) parameters from atm_flux( ) are the same here as well: nlayers and F_solar still represent the number of layers of atmosphere and incoming solar radiation, respectively. The debug feature is also the same: setting it to True when calling atm_flux_temp( ) will print out the indices and their corresponding values. Where this function differs is its additional parameters, emissivity and albedo. These are exactly as they are named, with emissivity representing the value of $\epsilon$, or the effectiveness of the atmosphere in emitting longwave radiation, and albedo being the reflectivity of the earth's surface. For the purposes of this lab, I set the albedo to be 0.33 for all experiments, as that is a generalized reasonable value for the earth.

This function first initializes the matrices in the exact same way as atm_flux( ), except this time, we have to take emissivity and albedo into account. Following the patterns found in class, I edited the corresponding diagonals in matrix A to include emissivity in the calculations. Additionally, the first element of matrix b has to be altered to account for the albedo, so I set that element to be -(F_solar / 4) * (1 - albedo). The influence of emitted flux from the other layers of the atmosphere also must be factored in, so I included multiple for loops in order to account for those additional impacts.

When calculating the resultant fluxes, we were instructed in the lab manual to use the inverse matrix technique. To do that, we can use some more built-in numpy functions. Passing A into np.linalg.inv( ) will invert it, which sets it up to be solved using np.matmul, making sure to pass in b for the second parameter of that function. These two lines give us the fluxes with emissivity and albedo now taken into account.

Finally, these resultant fluxes can be converted into temperatures, using the same method as flux2temp( ). However, the earth's surface temperature is calculated separately, because emissivity does not need to be taken into account, unlike the atmospheric layers. The same Stefan-Boltzmann equation is used, however in the atmospheric layer cases, emissivity is

multiplied by σ. All the temperatures are assigned to a dictionary, the labels changed accordingly, and are returned. I tested this function using the given values in lecture (0 layer atmosphere, incoming solar flux of 1350 W/m^2, emissivity value of 1.0, and an albedo of 0.33), and the result of about 251K matched what was determined in lecture.

Using the functions and creating figures

For Tasks 3-5, we were instructed to use our models to investigate different characteristics of the atmosphere and to interpret the results. The implementation for each of these tasks are listed below. All graphs and plots were produced using the values retrieved from using my functions and the matplotlib.pyplot library. Note that the analyses of the results are explained under the Analysis section of this report.

In Task 3, we performed two experiments. First, we found the emissivity that produces a certain surface temperature, and second, the number of atmospheric layers that produce that same temperature using the emissivity found in Experiment 1. The first experiment involved simply creating a range of emissivity values to investigate and plugging them into atm_flux_temp( ), using a for loop to do this efficiently. The resulting temperatures from this loop of calculations were stored in an array that was then plotted against emissivities, which allowed me to create an analysis of the results, described in the Analysis section. The second experiment was very similar to the first, however this time we were testing the number of layers it took to get the earth's surface to a certain temperature, assuming the emissivity was the value retrieved from Experiment 1. I used the exact same method for this part: create a range of atmospheric layer values, loop through each number and calculate the corresponding temperature using atm_flux_temp( ), and plot the results. For both experiments, I used np.interp( ) to show the matching variable value to the target temperature.The plot and analysis for Experiment 2 are also included in the Analysis section.

For Task 4, I also used a similar method as the experiments in Task 3. Starting with a specified range of atmospheric layers, this time for Venus, I used a for loop to calculate the corresponding surface temperatures using atm_flux_temp( ). The results were stored in an array, where I then used np.interp( ) to display the number of atmospheric layers needed to achieve a surface temperature of 700K.

Finally, for Task 5, I modified the code in atm_flux_temp( ) to account for the changes that come from a nuclear winter scenario. In a nuclear winter, the atmosphere blocks all incoming solar radiation from reaching the ground. Each variable is now different, so I renamed them to explicitly show that they are used for the nuclear winter situation (nlayers_winter, F_solar_winter, etc). This alteration impacts the value of the first element in matrix b. Matrix A is initialized the exact same way, this time with the nuclear winter version of emissivity, but b's first element must now be 0, and b[-1] is now -(F_solar_winter / 4) * (1 - albedo_winter). From here, the same procedure is followed as in atm_flux_temp( ), although the values are not automatically converted to temperatures. I used the resultant flux values from np.linalg( ) and passed them into flux2temp( ) to get the temperature values. Indexing into the correct element of

the temperature dictionary and printing it out reveals the surface temperature of the earth in a nuclear winter scenario.

## Analysis

Task 3

Figure 1 below shows the plot that represents the relationship between earth's surface temperature and atmospheric emissivity, given that there is one layer to the atmosphere. The positively sloped line indicates that with increasing emissivity comes an increasing surface temperature. This makes sense, because when the emissivity is greater, the atmosphere absorbs and re-emits a greater amount of longwave radiation. When the atmosphere is re-emitting more radiation, it's getting sent back down to the earth's surface, thereby warming the temperature there.

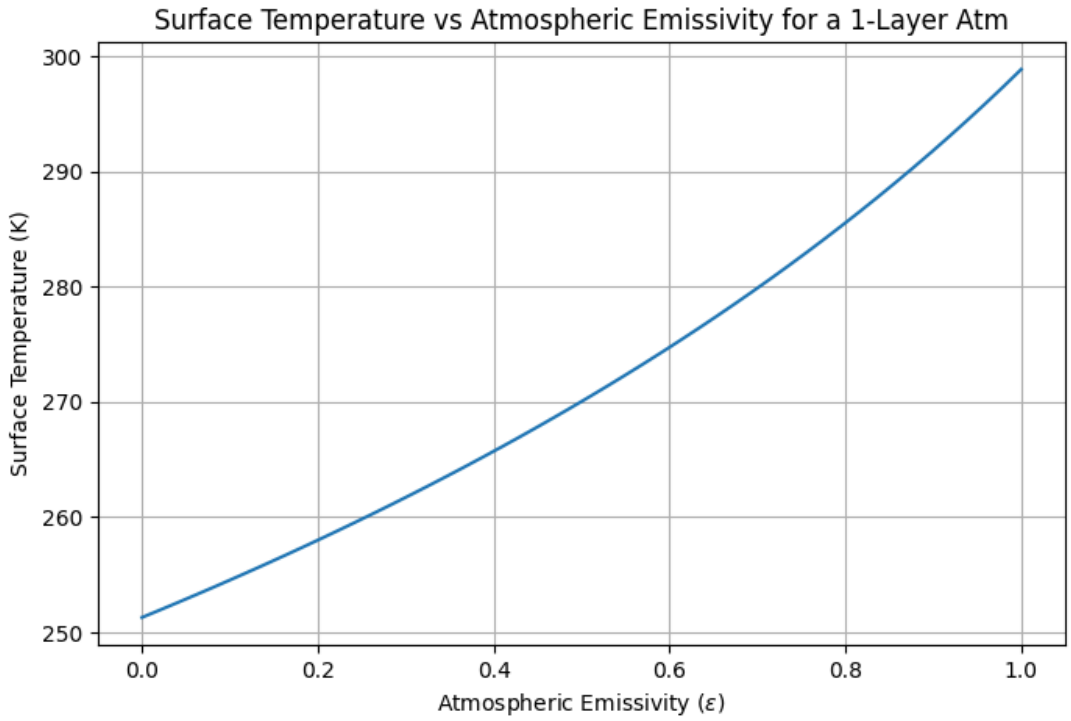



Figure 1

Using the coding technique described in the Methodology section (np.interp( )), the emissivity for a surface temp of 288K is 0.84. This is supported by the graph as well.

Next, I investigated the relationship between the number of atmospheric layers and the surface temperature. Figure 2 depicts this using the emissivity value of 0.84 found before, as instructed in the lab manual.

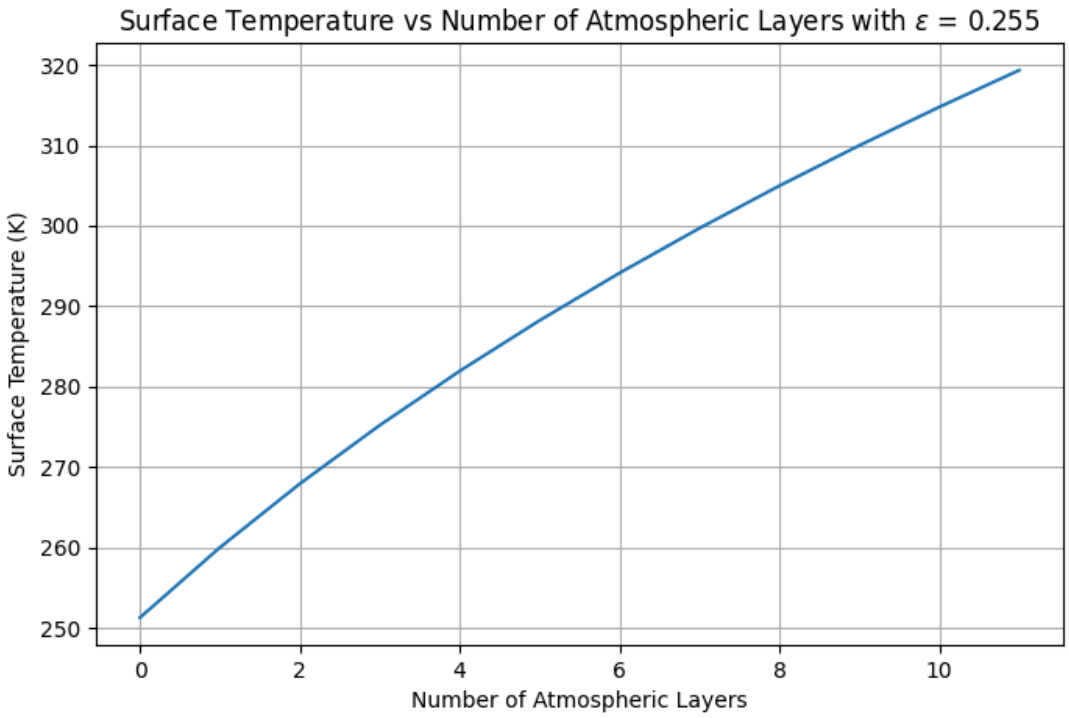



Figure 2

Based on the graph, another clearly positive trend is found. The surface temperature increases with every added atmospheric layer up. Based on the result from using np.linspace( ) and using the graph above, in order to get a surface temperature of 288K, about 4.96 layers of atmosphere are required.

Finally for this task, I plotted the altitude against temperature for a 4-layer atmosphere model (Figure 3) with an emissivity of 0.255. The graph reveals a decreasing trend in temperature as altitude increases. The decrease is sharper within the first layer of the atmosphere, then becomes a bit more gradual after that. Overall, if the surface temperature is around 280K in this case, by the fourth layer, the temperature has fairly steadily decreased to below 220K. The temperature decreasing makes sense because as altitude increases, less overall radiation is being absorbed and re-emitted as compared to at the surface.

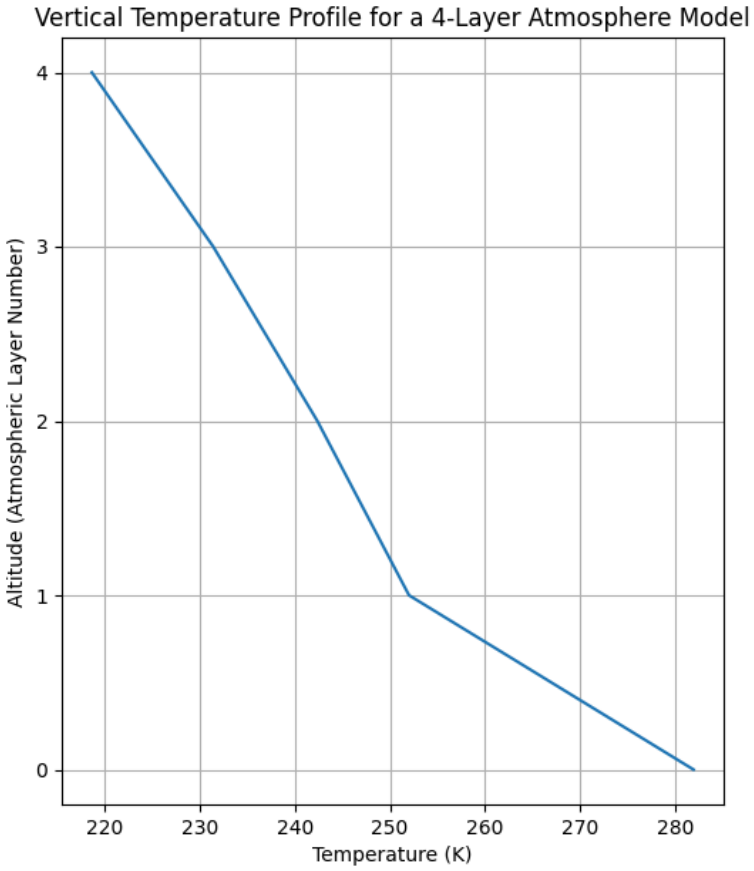



Figure 3

Task 4

For this task, I implemented my functions in a new context by investigating the atmospheric behavior of Venus. Given specified conditions for the characteristics of Venus's atmosphere and surface (surface temperature = 700K, solar flux = 2600 W/m^2, N>1 layers of atmosphere, emissivity = 1), I was able to plug those values into atm_flux_temp( ) and find the corresponding number of atmospheric layers required to achieve a surface temperature of 700K. The atmospheric layer amount corresponding to 700K is around 71.2 layers. This is a very hot temperature, so it makes sense that a lot of atmospheric layers are required to warm the surface of Venus up to that level.

Task 5

For the final task, I followed similar procedures as all the other tasks, but this time taking into account the different characteristics of the atmosphere. In a nuclear winter scenario, the surface of the earth doesn't receive any solar flux, and all warming is done purely by the radiative properties of the atmospheric layers. Given all of this, my model predicts a surface

temperature of about 227K in a nuclear winter scenario. Figure 4 shows an altitude vs temperature plot for this situation for a 5-layer atmosphere. Based on my model, the temperature is constant, holding steady at 227K all the way up to the fourth layer, before suddenly increasing in the topmost layer to 270K. This makes sense physically, because all the incoming solar radiation is being completely absorbed by that top layer. So, naturally, that layer will be hotter than all the others.

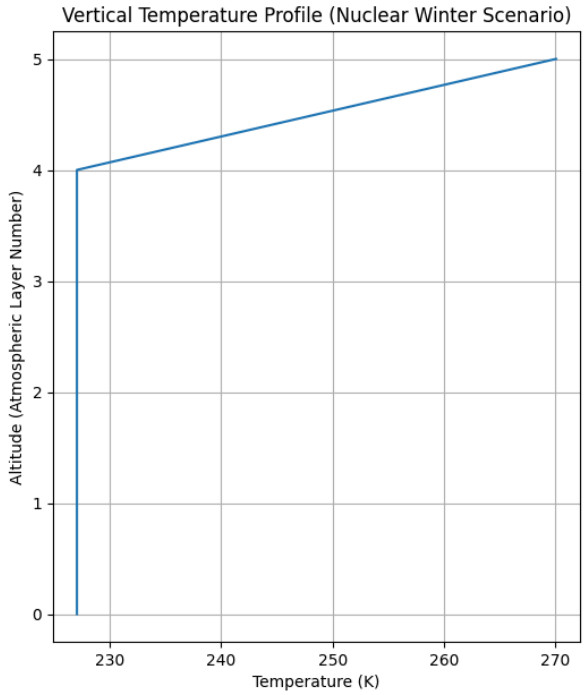



Figure 4